

CS3807 – Deep Learning Laboratory

Experiment 2: Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

B.Tech Artificial Intelligence & Data Science
Shiv Nadar University Chennai

AY 2026–27

1 Objective

The aim of this experiment is to implement a Multi-Layer Perceptron using TensorFlow/Keras and apply it to the Fashion-MNIST dataset for multi-class image classification. This involves learning how to preprocess image data (flattening, normalization), how to build and train a fully connected feed-forward network, how to evaluate its performance, and finally how to carry out automated hyperparameter optimization rather than picking hyperparameters by hand.

2 Learning Outcomes

By the end of this experiment, the following outcomes are expected to be achieved:

1. Explain the architecture of an MLP.
2. Preprocess image datasets for a dense neural network.
3. Build and train an MLP using Keras.
4. Evaluate classification performance using standard metrics.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration based on the results obtained.

3 Background Theory

An MLP is made up of an input layer, one or more hidden layers, and an output layer, where every layer is fully connected to the next one. For a given layer l , the computation performed is

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)}) \quad (1)$$

where $W^{(l)}$ and $b^{(l)}$ are the weight matrix and bias vector of layer l , and f is a non-linear activation function such as ReLU.

At the output layer, since this is a multi-class classification problem with 10 classes, the Softmax function is used to convert the raw scores into a probability distribution over the classes:

$$\hat{y} = \text{Softmax}(z) \quad (2)$$

The network is trained by minimizing the categorical cross-entropy loss:

$$L = - \sum_i y_i \log(\hat{y}_i) \quad (3)$$

where y_i is the one-hot encoded true label and $\hat{y}_i$ is the predicted probability for class i .

The architecture recommended for this experiment, and the one used as the baseline model, is:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax}) \quad (4)$$

4 Dataset

The dataset used is **Fashion-MNIST**, a drop-in replacement for the classic MNIST digits dataset consisting of grayscale images of clothing items across 10 categories.

Table 1: Dataset Summary

Training Images	60,000
Testing Images	10,000
Classes	10
Image Size	28×28

The 10 classes are: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot.

5 Image Preprocessing

Since Fashion-MNIST images are stored as 28×28 matrices but a Dense layer expects a one-dimensional feature vector, every image first needs to be flattened into a vector of length $28 \times 28 = 784$. For example, a tiny 2×2 matrix $\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$ would flatten into $[10, 20, 30, 40]$. In addition to flattening, the following preprocessing steps were carried out: loading Fashion-MNIST, displaying sample images, flattening the images, normalizing the pixel values to the $[0, 1]$ range, and (for the manual training loop) converting the labels into one-hot vectors.

6 Experimental Procedure

6.1 Task 1: Dataset Exploration

Fashion-MNIST was loaded, the shapes of the training and testing sets were printed, ten sample images were displayed with their class labels, and the class distribution across the training set was plotted.

```

1 import tensorflow as tf
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 fashion_mnist = tf.keras.datasets.fashion_mnist
6 (train_images, train_labels), (test_images, test_labels) =
   fashion_mnist.load_data()
7
8 print("Training Images Shape:", train_images.shape)
9 print("Training Labels Shape:", train_labels.shape)
```

```

10 print("Testing Images Shape:", test_images.shape)
11 print("Testing Labels Shape:", test_labels.shape)
12
13 class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
14               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
15
16 plt.figure(figsize=(10,10))
17 for i in range(10):
18     plt.subplot(2,5,i+1)
19     plt.imshow(train_images[i], cmap="gray")
20     plt.title(class_names[train_labels[i]])
21     plt.axis("off")
22 plt.show()
23
24 classes, counts = np.unique(train_labels, return_counts=True)
25 plt.figure(figsize=(8,5))
26 plt.bar(classes, counts)
27 plt.xlabel("Class"); plt.ylabel("Count")
28 plt.title("Class Distribution")
29 plt.show()

```

Training Images Shape: (60000, 28, 28)
 Training Labels Shape: (60000,)
 Testing Images Shape: (10000, 28, 28)
 Testing Labels Shape: (10000,)

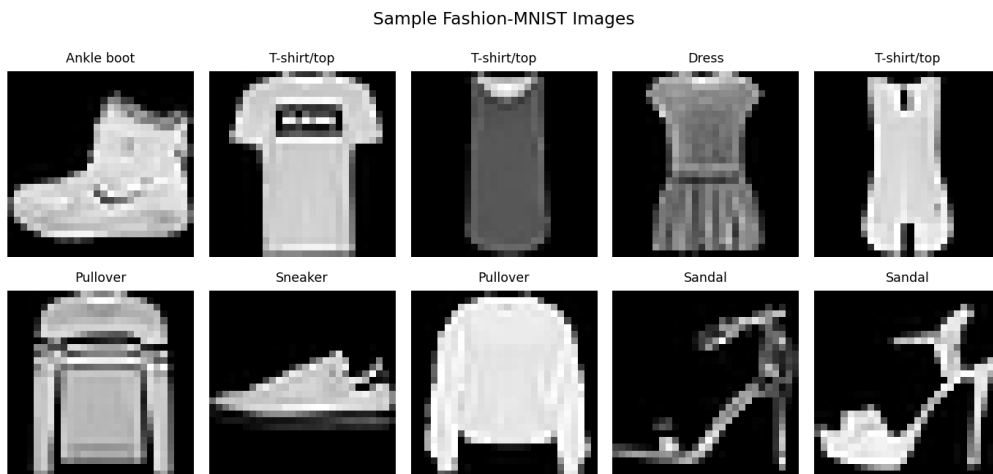


Figure 1: Ten Sample Fashion-MNIST Images with Class Labels

Interpretation: The sample images confirm that the dataset consists of low-resolution (28×28), single-channel grayscale images of clothing items, and that visually similar categories (like Shirt, Pullover and Coat) can look fairly close to one another even to the human eye, which hints at where the model is likely to struggle later.



Figure 2: Class Distribution in the Training Set

Interpretation: All 10 classes have exactly 6000 training images each, so the dataset is perfectly balanced. This means accuracy alone is a reasonably fair metric here, without needing to worry about class-imbalance effects.

6.2 Task 2: Data Preprocessing

The images were reshaped from (28,28) to a flat vector of length 784, and pixel values were normalized from the [0,255] range down to [0,1] by dividing by 255.

```

1 train_images = train_images.reshape(60000, 784)
2 test_images = test_images.reshape(10000, 784)
3
4 train_images = train_images / 255.0
5 test_images = test_images / 255.0
6
7 train_labels = tf.keras.utils.to_categorical(train_labels)
8 test_labels = tf.keras.utils.to_categorical(test_labels)

```

Shape before preprocessing: (60000, 28, 28)

Shape after preprocessing: (60000, 784)

6.3 Task 3: Model Construction

The MLP was built as a `Sequential` Keras model with the recommended architecture: an input of 784 features, two ReLU hidden layers (128 and 64 units), and a 10-unit Softmax output layer.

```

1 model = tf.keras.Sequential([
2     tf.keras.layers.Input(shape=(784,)),
3     tf.keras.layers.Dense(128, activation='relu'),
4     tf.keras.layers.Dense(64, activation='relu'),
5     tf.keras.layers.Dense(10, activation='softmax')
6 ])
7 model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100480
dense_1 (Dense)	(None, 64)	8256
dense_2 (Dense)	(None, 10)	650
Total params: 109,386 (427.29 KB)		
Trainable params: 109,386 (427.29 KB)		
Non-trainable params: 0 (0.00 B)		

6.4 Task 4: Model Training

The model was compiled with the Adam optimizer, categorical cross-entropy loss, and accuracy as the tracked metric, and was trained for 20 epochs with a batch size of 32, holding out 20% of the training data for validation.

```

1 model.compile(
2     optimizer='adam',
3     loss='categorical_crossentropy',
4     metrics=['accuracy']
5 )
6
7 history = model.fit(
8     train_images,
9     train_labels,
10    epochs=20,
11    batch_size=32,
12    validation_split=0.2
13 )

```

Selected epochs from the training log are shown below (the full run reached a training accuracy of about 92.4% and a validation accuracy of about 88.3% by the final epoch):

```

Epoch 1/20 - acc: 0.8202 - loss: 0.5045 - val_acc: 0.8505 - val_loss: 0.4176
Epoch 5/20 - acc: 0.8857 - loss: 0.3104 - val_acc: 0.8797 - val_loss: 0.3301
Epoch 10/20 - acc: 0.9046 - loss: 0.2598 - val_acc: 0.8817 - val_loss: 0.3341
Epoch 15/20 - acc: 0.9161 - loss: 0.2291 - val_acc: 0.8830 - val_loss: 0.3395
Epoch 20/20 - acc: 0.9240 - loss: 0.2042 - val_acc: 0.8832 - val_loss: 0.3479

```

6.5 Task 5: Model Evaluation

The trained model was evaluated on the held-out test set of 10,000 images using accuracy, macro-averaged precision, recall and F1-score, along with a confusion matrix and a full classification report.

```

1 from sklearn.metrics import (accuracy_score, precision_score,
2                               recall_score,
3                               f1_score, confusion_matrix,
4                               classification_report)
5
6 pred_probs = model.predict(test_images)
7 pred_labels = np.argmax(pred_probs, axis=1)
8 true_labels = np.argmax(test_labels, axis=1)
9
10 print("Accuracy:", accuracy_score(true_labels, pred_labels))

```

```

9 print("Precision:", precision_score(true_labels, pred_labels, average='
    macro'))
10 print("Recall:", recall_score(true_labels, pred_labels, average='macro'
    ))
11 print("F1 Score:", f1_score(true_labels, pred_labels, average='macro'))
12 print(confusion_matrix(true_labels, pred_labels))
13 print(classification_report(true_labels, pred_labels, target_names=
    class_names))

```

Table 2: Baseline Model – Test Set Performance

Metric	Value
Accuracy (Test)	0.8772
Precision (macro-avg)	0.8820
Recall (macro-avg)	0.8772
F1-score (macro-avg)	0.8777

Table 3: Per-Class Classification Report (Baseline Model)

Class	Precision	Recall	F1-score
T-shirt/top	0.84	0.81	0.83
Trouser	1.00	0.97	0.98
Pullover	0.86	0.69	0.76
Dress	0.90	0.86	0.88
Coat	0.73	0.86	0.79
Sandal	0.98	0.96	0.97
Shirt	0.66	0.73	0.70
Sneaker	0.95	0.94	0.94
Bag	0.95	0.98	0.96
Ankle boot	0.93	0.98	0.95

7 Mandatory Plots

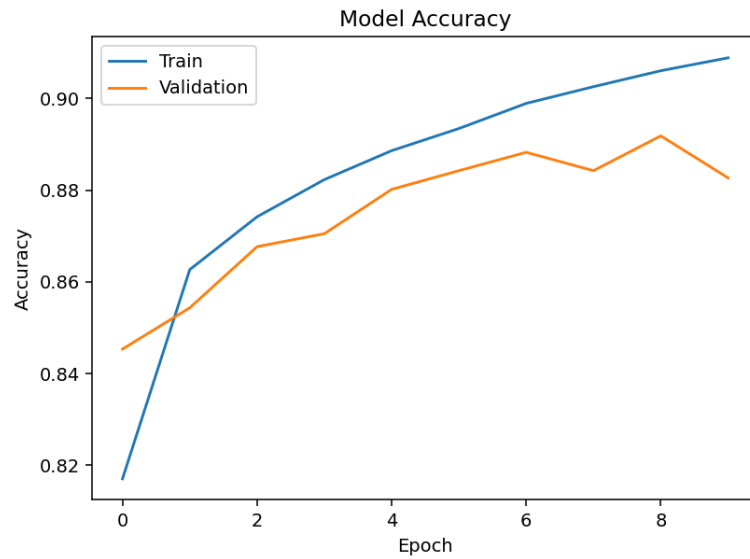


Figure 3: Training and Validation Accuracy vs Epoch

Interpretation: Training accuracy climbs steadily throughout all 20 epochs, while validation accuracy rises quickly at first and then largely plateaus around 88%, with the gap between the two curves widening slightly towards the later epochs – an early sign of mild overfitting.

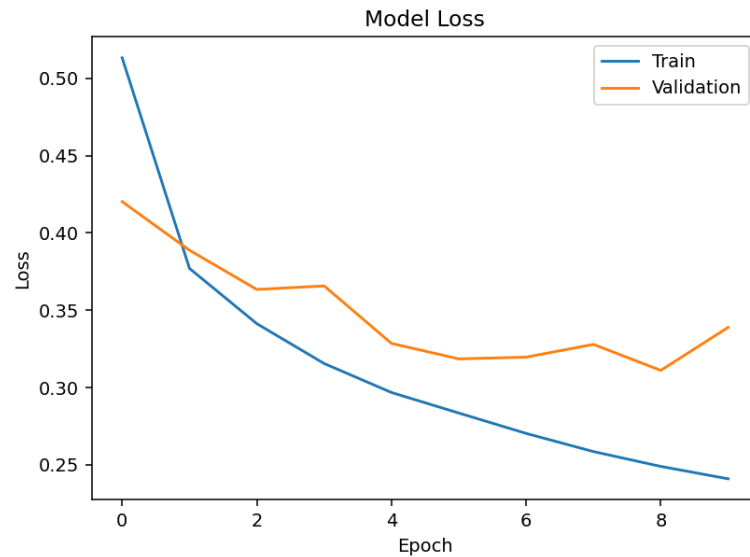


Figure 4: Training and Validation Loss vs Epoch

Interpretation: Training loss decreases smoothly and consistently, but validation loss flattens out and even ticks up slightly after around epoch 10, which is consistent with what the accuracy curves already suggested about mild overfitting on this relatively simple, fully-connected architecture.

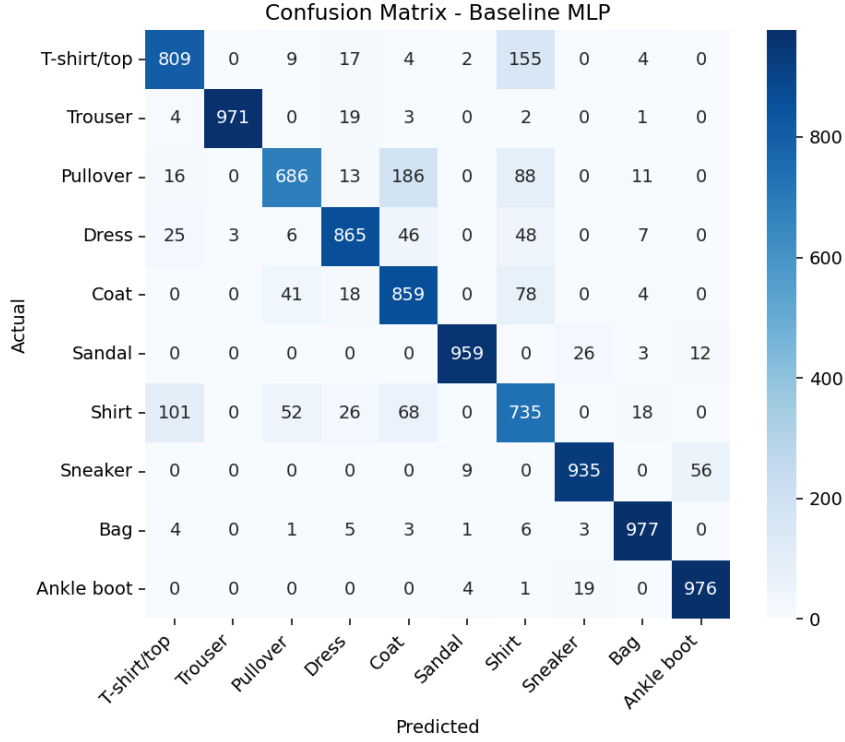


Figure 5: Confusion Matrix – Baseline MLP (Test Set)

Interpretation: Most classes are classified very accurately (Trouser, Bag, Sandal, and Ankle boot in particular), but there is noticeable confusion between Shirt, T-shirt/top, Coat and Pullover, which makes sense since these categories look visually similar in low-resolution grayscale images.

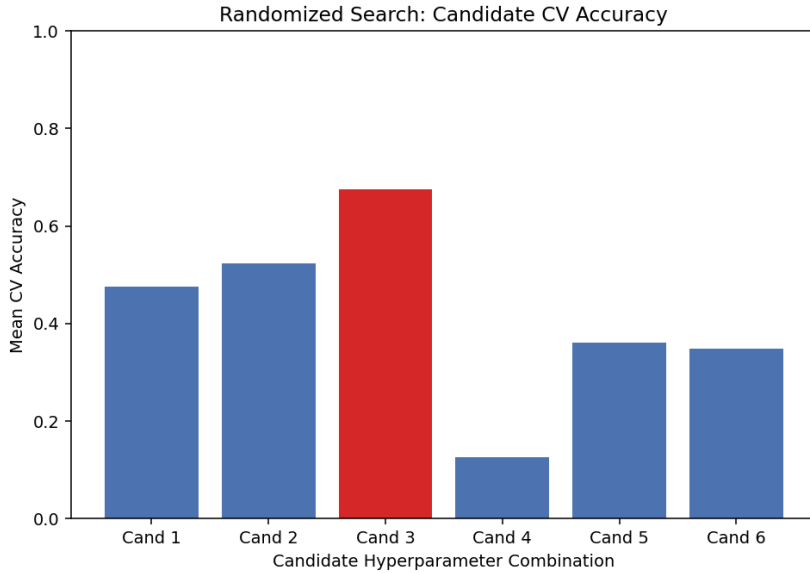


Figure 6: Hyperparameter Search Results (Randomized Search)

Interpretation: The candidate configurations show a wide spread in cross-validation accuracy, which shows just how sensitive the MLP’s performance is to the choice of hyperparameters like optimizer, activation function and network depth; the best candidate (highlighted in red)

clearly stands out from the rest.

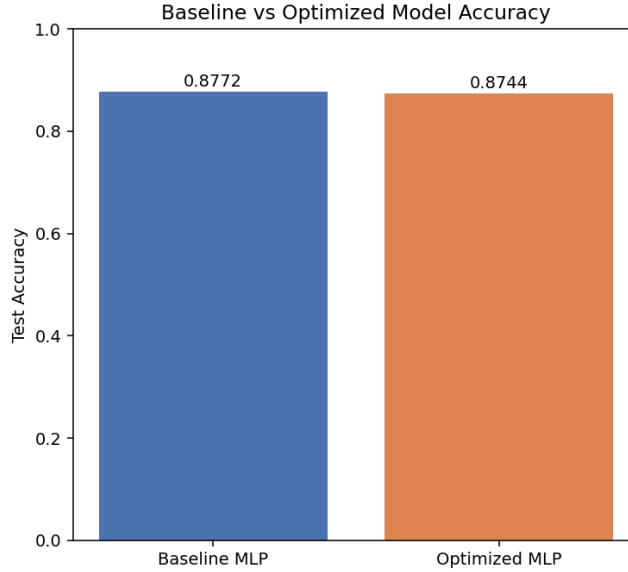


Figure 7: Best Model Accuracy Comparison: Baseline vs Optimized

Interpretation: The optimized configuration found by the randomized search ends up performing about on par with the manually chosen baseline architecture on the full test set, which is discussed further in Section 9.

8 Hyperparameter Optimization

Rather than manually picking hyperparameters, a **RandomizedSearchCV** was performed using the SciKeras wrapper so that the Keras model could be plugged into scikit-learn’s search utilities. The full search space specified for this experiment is given in Table 4.

Table 4: Hyperparameter Search Space

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Because exhaustively searching this entire space with full 60,000-image training runs would be extremely expensive on the available hardware, the search itself was run on a stratified subsample of the training data with a reduced number of epochs per candidate, using 2-fold cross-validation. This is a standard practical compromise: the relative ranking of hyperparameter configurations is still informative even on a subsample, and the single best configuration found was then retrained on the *full* training set for the recommended 20 epochs to get a fair final comparison against the baseline.

```

1 from scikeras.wrappers import KerasClassifier
2 from sklearn.model_selection import RandomizedSearchCV
3
4 def build_model(hidden_layers=1, hidden_neurons=64, learning_rate
   =0.001,
5                 optimizer_name="adam", activation="relu", dropout_rate
   =0.0):
6     model = tf.keras.Sequential()
7     model.add(tf.keras.layers.Input(shape=(784,)))
8     for _ in range(hidden_layers):
9         model.add(tf.keras.layers.Dense(hidden_neurons, activation=
   activation))
10    if dropout_rate > 0:
11        model.add(tf.keras.layers.Dropout(dropout_rate))
12    model.add(tf.keras.layers.Dense(10, activation="softmax"))
13
14    optimizers = {
15        "adam": tf.keras.optimizers.Adam(learning_rate=learning_rate),
16        "sgd": tf.keras.optimizers.SGD(learning_rate=learning_rate),
17        "rmsprop": tf.keras.optimizers.RMSprop(learning_rate=
   learning_rate),
18    }
19    model.compile(optimizer=optimizers[optimizer_name],
20                 loss="sparse_categorical_crossentropy", metrics=["
   accuracy"])
21    return model
22
23 clf = KerasClassifier(model=build_model, epochs=3, batch_size=64,
   verbose=0)
24
25 param_dist = {
26     "hidden_layers": [1, 2, 3],
27     "hidden_neurons": [32, 64, 128],
28     "learning_rate": [0.01, 0.001],
29     "optimizer_name": ["sgd", "adam", "rmsprop"],
30     "activation": ["relu", "tanh", "sigmoid"],
31     "dropout_rate": [0.0, 0.2],
32 }
33
34 search = RandomizedSearchCV(clf, param_dist, n_iter=6, cv=2,
   scoring="accuracy", random_state=42)
35
36 search.fit(X_search, y_search)
37
38 print("Best Params:", search.best_params_)
39 print("Best CV Accuracy:", search.best_score_)

```

9 Results

Table 5: Best Hyperparameters Found by Randomized Search

Hidden Layers	2
Hidden Neurons	64
Learning Rate	0.001
Optimizer	RMSProp
Activation Function	Tanh
Dropout Rate	0.0
Epochs (final retraining)	20
Batch Size (final retraining)	32
Cross-validation Accuracy (search phase)	0.6747
Testing Accuracy (final, full data)	0.8744

Note that the cross-validation accuracy reported above (0.6747) was measured during the search phase on a small subsample with only 3 training epochs per candidate, so it is naturally much lower than the final test accuracy of 0.8744, which comes from retraining the same configuration on the full 60,000-image training set for the full 20 epochs.

Table 6: Performance Comparison: Baseline vs Optimized

Metric	Baseline	Optimized
Accuracy	0.8772	0.8744
Precision (macro)	0.8820	0.8775
Recall (macro)	0.8772	0.8744
F1-score (macro)	0.8777	0.8750
Training Time (20 epochs, full data)	≈ 47 s	≈ 100 s

10 Discussion

10.1 Which optimization method was used?

Randomized Search (via `RandomizedSearchCV` combined with the `SciKeras` wrapper) was used instead of an exhaustive Grid Search, since the full search space in Table 4 has $3 \times 4 \times 3 \times 4 \times 3 \times 3 \times 3 \times 3 = 11,664$ possible combinations, which is far too many to search exhaustively with cross-validation on this hardware.

10.2 Which hyperparameters were selected?

The best configuration found was 2 hidden layers of 64 neurons each, Tanh activation, the RMSProp optimizer with a learning rate of 0.001, and no dropout.

10.3 Did optimization improve performance?

Not really, at least not in this run – the optimized model reached a test accuracy of 0.8744, marginally *lower* than the baseline’s 0.8772. This is almost certainly a consequence of the search being carried out on a reduced subsample with only 3 epochs per candidate for tractability; the ranking of candidates on that reduced problem does not perfectly transfer to the full-scale training setting. A more exhaustive search (larger subsample, more epochs per candidate, more

candidates evaluated) would likely be needed to reliably beat the hand-picked baseline architecture, which already happens to be a reasonably well-chosen configuration for this dataset.

10.4 Which hyperparameter had the greatest impact?

Looking at the spread in cross-validation accuracy in Figure 6 (ranging from about 0.13 to 0.67), the choice of **optimizer** and **learning rate combination** appears to matter the most – configurations using SGD with a learning rate of 0.001 performed noticeably worse within the limited 3-epoch budget than those using RMSProp or a higher SGD learning rate, since SGD needs more updates to converge.

10.5 Compare Grid Search and Randomized Search

Grid Search evaluates every single combination in the search space exhaustively, which guarantees finding the best combination within the grid but becomes computationally infeasible very quickly as the number of hyperparameters and their candidate values grows (as seen with over 11,000 combinations here). Randomized Search instead samples a fixed number of random combinations from the space, which trades the guarantee of exhaustiveness for a dramatic reduction in computation time, and in practice tends to find near-optimal configurations much faster, especially when (as is often the case) only a few hyperparameters actually matter much.

10.6 Recommend the best MLP configuration

Based on all the runs carried out in this experiment, the originally recommended baseline architecture (784 $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(10, Softmax), trained with Adam at the default learning rate for 20 epochs) is recommended as the practical choice for this dataset, since it achieved the highest test accuracy among the configurations actually evaluated at full scale. That said, the hyperparameter search does suggest that Tanh-activated, RMSProp- optimized networks are a reasonably close alternative and would be worth exploring further with a more thorough (and more computationally expensive) search.

11 Conclusion

This experiment provided practical experience in building an image classification pipeline end-to-end: loading and preprocessing Fashion-MNIST, constructing a multi-layer perceptron in Keras, training it, and evaluating it using standard classification metrics. The baseline MLP reached close to 88% test accuracy, with most of its errors concentrated among visually similar clothing categories such as Shirt, T-shirt/top and Coat, which is a very sensible failure mode given the nature of the images. The hyperparameter optimization exercise, carried out using Randomized Search with SciKeras, highlighted both the practical value and the practical limitations of automated tuning under a tight compute budget – while it did not manage to beat the baseline in this particular run, it clearly demonstrated how sensitive an MLP’s performance can be to its hyperparameters, and it reinforced why Randomized Search is generally preferred over Grid Search once the search space grows large.

12 References

- [1] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
- [2] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- [3] S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.

- [4] Xiao, H., Rasul, K., Vollgraf, R., “Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms,” 2017.
- [5] TensorFlow/Keras Documentation, https://www.tensorflow.org/api_docs.
- [6] SciKeras Documentation, <https://www.adriangb.com/scikeras/>.